Web Dev Mid-Term: Important Questions & Answers

Node.js + Express (Unit 1 and Unit 2). DBMS removed, duplicate questions merged.

Q1. What is web development? What is backend development and its uses?

Web development is building websites and web applications. It has two sides: frontend (what the user sees) and backend (the server side).

Backend development refers to the server-side of an application. It is responsible for:

- Processing user requests
- Performing business logic
- Authentication
- Sending responses to users

Examples: login system, order management, sending OTP.

Q2. What is the purpose of HTTP? What are its methods?

HTTP (Hyper Text Transfer Protocol) is the foundation of web communication. It defines how messages are transmitted between clients and servers.

Method	Use
GET	Retrieve data
POST	Create a new resource
PUT	Replace an entire resource
PATCH	Partially update a resource
DELETE	Remove a resource

Q3. HTTP request vs HTTP response (parts)

HTTP request: a message sent by the client to the server. It is used to request data or perform an action. It contains:

- `req.method` : GET, POST, etc.
- `req.url` : requested URL
- `req.params` : URL parameters
- `req.query` : query string values
- `req.body` : data sent by the client

HTTP response: the message sent by the server back to the client. It contains the result of the request.

Request	Response
Sent by client	Sent by server
Asks for data or an action	Gives the result
Has method, URL, headers, body	Has status code, headers, body

Common response methods in Express:

- `res.send()` : sends a response
- `res.json()` : sends JSON data
- `res.status()` : sets the status code
- `res.sendFile()` : sends a file

Q4. What is Node.js? Why is it called a JavaScript environment?

Node.js is a runtime environment that allows JavaScript to run outside the browser. It is used for server-side scripting, building APIs, backends and services, and real-time apps. It is called a JS environment because it provides everything JavaScript needs to run on a server (engine, file access, networking) without a browser.

Q5. What is npm?

npm (Node Package Manager) is the world's largest software library for JavaScript. It helps developers to:

- Install packages
- Update packages
- Remove packages
- Share their own packages

```
npm install express
```

Q6. What are modules? Types of modules

Modules are self-contained units of code. Each file in Node.js is a separate module with private scope.

Benefits: reusability, organization, scope isolation, encapsulation.

Types:

1. **Core modules:** built into Node.js, no installation needed. Example: `fs`, `http`, `os`, `path`.
2. **Local modules:** files you create in your project, imported with a relative path. Example: `require('./Mymodule')`.
3. **Third-party modules:** installed via npm from the npm registry. Example: `express`, `axios`.

Q7. Core modules: the OS module and its methods

The `os` (Operating System) module gives information about your computer.

```
const os = require('os');

console.log(os.platform());   // computer platform
console.log(os.arch());       // CPU architecture
console.log(os.type());       // operating system type
console.log(os.freemem());    // free memory
console.log(os.totalmem());   // total memory
```

Q8. File System (fs) module: all operations (sync and async)

The `fs` module is used to work with files.

- Read file: read data from a file
- Write file: create a new file or replace existing content
- Append file: add new data at the end of an existing file
- Unlink file: delete a file

Asynchronous (callback based, non-blocking)

```
const fs = require('fs');

// 1. Read
fs.readFile('file.txt', 'utf-8', (err, data) => {
  if (err) {
    console.log(err);
  } else {
    console.log(data);
  }
});

// 2. Write (replaces content)
fs.writeFile('file.txt', 'hello students', (err) => {
  if (err) {
    console.log(err);
  }
});

// 3. Append (adds at end)
fs.appendFile('file.txt', '\nhello students', (err) => {
  if (err) {
    console.log(err);
  }
});

// 4. Delete
fs.unlink('file1.txt', (err) => {
  if (err) {
    console.log(err);
  }
});
```

Synchronous (blocking, waits for each step)

```
const fs = require('fs');

const data = fs.readFileSync('file.txt', 'utf-8');
console.log(data);

fs.writeFileSync('file.txt', 'hello students');
fs.appendFileSync('file.txt', '\nhello students');
fs.unlinkSync('file1.txt');
```

Sync vs async example

```
console.log('first');
fs.readFile('file.txt', 'utf-8', (err, data) => {
  console.log(data);
});
console.log('second');
```

Output order: `first`, `second`, then the file data (because `readFile` is async and does not block).

Q9. What is process.argv?

`process.argv` is an array that contains the command-line arguments passed when Node.js was started. It is how you make your scripts configurable from the terminal.

```
// app.js
console.log(process.argv);
```

```
node app.js hello 5
```

- `process.argv[0]`: path of node
- `process.argv[1]`: path of the file
- `process.argv[2]`: `hello`
- `process.argv[3]`: `5`

Difference between os module and process.argv: the `os` module gives information about your computer (for example the platform), while `process.argv` gives the arguments passed to your script.

Q10. Create an HTTP server with 2-3 URLs using if/else, 404 in else, with listen

An HTTP server receives HTTP requests from a client and sends back an HTTP response.

```
// server.js
const http = require('http');

const users = [
  { id: 1, name: 'Muskan', email: 'abc@gmail.com' },
  { id: 2, name: 'Harshi', email: 'xyz@gmail.com' }
];

const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<h1>Welcome to Home Page</h1>');
    res.end();
  } else if (req.url === '/users' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.write(JSON.stringify(users));
    res.end();
  } else if (req.url === '/about' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<h1>About Page</h1>');
    res.end();
  } else {
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.write('<h1>Page Not Found</h1>');
    res.end();
  }
});

server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Note: in your notebook the not-found branch uses status `400`. The correct code for Page Not Found is **404**, so write 404 in the exam.

Q11. GET and POST method (10 marks)

GET retrieves data. **POST** creates a new resource, sending data in the request body.

Using the http module

```
const http = require('http');

const users = [{ id: 1, name: 'Muskan' }];

const server = http.createServer((req, res) => {
  // GET: send all users
  if (req.url === '/users' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(users));

  // POST: add a new user
  } else if (req.url === '/users' && req.method === 'POST') {
    let body = '';

    req.on('data', (chunk) => {
      body += chunk;
    });

    req.on('end', () => {
      const newUser = JSON.parse(body);
      users.push(newUser);
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify(newUser));
    });

  } else {
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end('<h1>Page Not Found</h1>');
  }
});

server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Using Express

```
const express = require('express');
const app = express();

app.use(express.json());

const users = [{ id: 1, name: 'Muskan' }];

app.get('/users', (req, res) => {
  res.json(users);
});

app.post('/users', (req, res) => {
  const newUser = req.body;
  users.push(newUser);
  res.status(201).json(newUser);
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Test POST with Postman.

Postman is one of the most popular API testing tools. It allows developers to send HTTP requests, view responses, and test APIs without a frontend.

Q12. Express.js vs HTTP module

Express.js is a fast, lightweight and flexible web framework for Node.js that helps developers build web applications and APIs.

Express.js	Node HTTP module
Less code	More code
Simple routing	Manual routing
Easy to scale	Difficult to scale
Beginner friendly	Not beginner friendly

Q13. How to make an Express server (10 marks)

Step 1: create a project and install Express.

```
npm init -y
npm install express
```

Step 2: create `server.js`.

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello From Server');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Step 3: run it.

```
node server.js
```

Step 4: open `http://localhost:3000` in the browser.

Q14. CRUD in Express (also works the same idea in the http server)

CRUD = Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE).

```
const express = require('express');
const app = express();

app.use(express.json());

let students = [
  { id: 1, name: 'Muskan' },
```

```
  { id: 2, name: 'Harshi' }
];

// READ all
app.get('/students', (req, res) => {
  res.json(students);
});

// READ one
app.get('/students/:id', (req, res) => {
  const student = students.find((s) => s.id === Number(req.params.id));
  if (!student) {
    return res.status(404).send('Student not found');
  }
  res.json(student);
});

// CREATE
app.post('/students', (req, res) => {
  const newStudent = { id: students.length + 1, name: req.body.name };
  students.push(newStudent);
  res.status(201).json(newStudent);
});

// UPDATE
app.put('/students/:id', (req, res) => {
  const student = students.find((s) => s.id === Number(req.params.id));
  if (!student) {
    return res.status(404).send('Student not found');
  }
  student.name = req.body.name;
  res.json(student);
});

// DELETE
app.delete('/students/:id', (req, res) => {
  students = students.filter((s) => s.id !== Number(req.params.id));
  res.send('Student deleted');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Q15. req.params vs req.query

req.params: the `req.params` object is used to access Route Parameters, which are dynamic values present inside the URL. Real-world example: `/students/101`, `/students/102`.

req.query: query parameters are values sent in the URL after the `?` symbol. They are used to send extra information to the server without changing the route. Example: `/students?course=MERN`.

req.params	req.query
Part of the route path	After `?` in the URL
Defined in route: `/students/:id`	Not defined in route
`/students/101`	`/students?course=MERN`
`req.params.id`	`req.query.course`

```
app.get('/students/:id', (req, res) => {
  res.send(req.params.id);
});

app.get('/students', (req, res) => {
  res.send(req.query.course);
});
```

Q16. Static route vs dynamic route

Static route: the URL remains fixed and is the same for every request. It does not use route parameters. Example: `/about`. Easier to create and use.

Dynamic route: the URL can change and contains a variable value. It uses route parameters like `:id`. Example: `/user/:id`.

Q17. MVC architecture (main folders)

MVC = Model, View, Controller. It separates the application into three parts so the code stays organized.

```
project/
├── models/         (data and database logic)
├── views/          (what the user sees: HTML/UI)
├── controllers/    (logic that handles requests)
├── routes/         (URL paths mapped to controllers)
├── middleware/     (custom middleware)
├── server.js       (starts the app)
└── package.json
```

- **Model:** handles data and the database.
- **View:** the user interface.
- **Controller:** receives the request, uses the model, and sends the response.

Q18. Define controller

A controller is a function (or file of functions) that contains the logic for handling a request. It receives `req` and `res`, talks to the model if needed, and sends back the response. Routes stay clean because the logic lives in the controller.

```
// controllers/studentController.js
exports.getStudents = (req, res) => {
  res.json([{ id: 1, name: 'Muskan' }]);
};
```

Q19. Define routing

Routing decides how an application responds to a client request for a specific URL (endpoint) and HTTP method. In Express it looks like `app.METHOD(path, handler)`.

```
// routes/studentRoutes.js
const express = require('express');
const router = express.Router();
const { getStudents } = require('../controllers/studentController');

router.get('/students', getStudents);

module.exports = router;
```

```
// server.js
const studentRoutes = require('./routes/studentRoutes');
app.use('/', studentRoutes);
```

Q20. Middleware (very important: code for 10 marks, types for 2 marks)

Definition: middleware is a function that runs between the client request and the server response. It can perform tasks like logging, authentication, validation, and error handling. It has access to `req`, `res` and `next`. Calling `next()` passes control to the next middleware or route.

Code: custom log middleware + API key middleware + morgan

```
// server.js
const express = require('express');
const morgan = require('morgan');
const app = express();
const PORT = 3000;

// third-party middleware
app.use(morgan('dev'));

// custom log middleware: prints method, url and time
const logMiddleware = (req, res, next) => {
  req.data = 'this is data from middleware';
  console.log('Request URL:', req.url, 'Method:', req.method, 'Time:', new
Date().toLocaleString());
  next();
};

// custom authentication middleware
const apiMiddleware = (req, res, next) => {
  const API_KEY = req.query.API_KEY;
  if (API_KEY !== '1234') {
    return res.send('API KEY is not valid');
  }
  console.log('authenticated');
  next();
};
```

```
app.use(logMiddleware);   // runs for every route

app.get('/', (req, res) => {
  console.log('Home Page');
  console.log('Data from middleware:', req.data);
  res.send('Hello From Server');
});

// apiMiddleware runs only for this route
app.get('/weather-data', apiMiddleware, (req, res) => {
  res.json({
    city: 'New Delhi',
    temperature: 32,
    weather: 'sunny'
  });
});

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Test: `http://localhost:3000/weather-data?API_KEY=1234`

Note: in your notebook, `res.send('API KEY is not valid')` is followed by `console.log` and `next()`. Add `return` before `res.send` (as above), otherwise the code keeps running after sending the response.

Types of middleware (2 marks)

1. **Application-level:** used with `app.use()` and runs for every request or a path.
2. **Router-level:** works like application-level but bound to `express.Router()`.
3. **Built-in:** comes with Express. Example: `express.json()`, `express.static()`.
4. **Third-party:** installed via npm. Example: `morgan`, `cors`.
5. **Error-handling:** has four parameters `(err, req, res, next)`.

Q21. Error handling using try-catch

`try-catch` is used to catch errors during execution. It is especially useful for:

- Database operations

- Async functions
- External API calls

The `catch` block catches errors inside the `try` block, especially for asynchronous operations like database queries.

```
app.get('/data', async (req, res) => {
  try {
    const result = await someDatabaseCall();
    res.json(result);
  } catch (err) {
    console.log(err);
    res.status(500).send('Something went wrong');
  }
});
```

Q22. Common response methods

- `res.send()` : sends a response
- `res.json()` : sends JSON data
- `res.status()` : sets the status code
- `res.sendFile()` : sends a file

Notes about this list

- Merged duplicates: "HTTP request and/vs response" and "request vs response" are now one answer (Q3).
- Not included: "lect 10 imp" in your PDF is unclear, so I did not answer it. Check what it refers to (maybe lecture 10 or an important topic in lecture 10).
- Also corrected two small notebook mistakes: 404 instead of 400, and `return` in the API key middleware.